

Government College Women University

Faisalabad Campus



Software Engineering

Assignment no 1

Name:

Hiba Noor

Roll No:

2505

Submitted to:

Ms.Sadia Saher

SDLC Models Comparison Table

Model Names	Features	Advantages	Disadvantages	Applications
Waterfall Model	• Linear sequence: Each phase must finish completely before the next begins • Heavy documentation: Formal records and deliverables are required at every stage. • Low flexibility: Extremely difficult and costly to accommodate changes once work starts.	• High simplicity: Simple, straightforward, and easy to manage throughout. • Clear visibility: Every stage produces well-defined, documented deliverables. • Stable requirements: Highly effective when project goals are fixed and fully known.	• Low flexibility: Extremely rigid when handling changing customer requirements after development starts. • Premature freezing: Locking specifications too early often results in software that misses true user needs. • Delayed delivery: Working software is completely unavailable until the very final stages of the lifecycle.	• Complex systems: Large projects with well-understood hardware and software parts. • Legacy replacements: Rewriting existing systems where all requirements are fixed. • Regulated domains: Critical systems requiring upfront compliance (e.g., avionics).
V-Model	• Sequential extension: Arranges traditional Waterfall phases into a structured "V" shape. • Phase mapping: Pairs every development stage directly with a corresponding test phase. • Early planning: Defines all verification and validation tests well before coding starts.	• Parallel testing: Integrates test planning directly alongside early development stages. • High traceability: Links each architecture phase to its specific verification test. • Early detection: Catches major defects early through upfront review before coding.	• Rigid progression: Inherits the strict, sequential inflexibility of the Waterfall model. • Costly alterations: Extremely expensive and difficult to manage scope changes late. • Heavy documentation: Extensive documentation requirements can easily delay project delivery.	• Safety-critical applications: Embedded systems needing early validation (e.g., medical devices, ABS) • High-traceability domains: Aerospace and defense projects requiring strict compliance audits. • Contract-driven products: Client solutions requiring formal proof for every initial requirement.

Iterative Model	• Cyclic refinement: Builds a simple version first, then improves it each round. • Continuous evolution: Upgrades the main system and features with every pass. • Feedback-driven: Uses regular user reviews to make the whole project better.	• Iterative refinement: Makes the project better step-by-step in each round. • Early validation: Tests simple working models early to get quick feedback. • Agile adjustments: Fixes major mistakes easily before they become bigger problems.	• Scope creep: Unchecked iterations make total costs and deadlines hard to forecast. • Architectural degradation: Rushed code revisions can quickly compromise overall system design. • High overhead: Demands constant customer involvement to review every single release	• Experimental research: Tests new ideas and algorithms where needs can change anytime • Usability testing: Uses quick trial models to see how easily people can use the system. • Simulation software: Builds complex tools that need constant tuning based on user advice.
Incremental Model	• Modular breakdown: Breaks the big project into smaller, easy-to-handle parts. • Priority delivery: Builds and releases the most important features first. • Cumulative expansion: Adds new pieces step-by-step on top of what already works.	• Lower rework: Saves time and money when customers change their minds. • Rapid delivery: Gives users working parts of the software very quickly.. • Targeted quality: Tests the most important features first and most thoroughly.	• Low visibility: Hard for bosses to track real progress because reporting costs too much. • Structural degradation: The system gets messy and breaks down without regular cleanups. • Hidden shared needs: Hard to know early on which tools all parts will share.	• E-commerce platforms: Commercial services where rapid time-to-market offers immediate competitive value. • Web applications: Fast-paced online products where user needs shift quickly across releases. • Enterprise software: Internal business tools where practical staff feedback guides future modules.
Reuse-Oriented Model	• Component assembly: Uses ready-made software parts instead of coding from scratch. • Flexible specifications: Adjusts project goals to match what available parts can do.	• Accelerated delivery: Finishes the software much faster so it launches sooner. • Cost efficiency: Saves a lot of money and lowers project risks. • Reduced codebase: Needs way less code to write, test, and fix.	• Requirement compromises: Forces you to give up ideal features to fit what the ready-made parts allow. • Loss of control: Leaves you stuck depending on outside companies for updates and fixes. • Imperfection penalty: Ready-made parts rarely fit together smoothly without extra patching work.	• ERP implementations: Big business systems built by setting up existing platforms like SAP. • API-driven services: Websites and apps made by linking ready-made online services together. • COTS-based systems: Standard office tools built around ready-made accounting or database software.

Boehm's Spiral Model	• Risk-driven spiral: Moves in loops, checking and handling major risks at every step.. • Prototyping focus: Builds early test models to clear up doubts before moving forward. • High-stakes orientation: Made for huge, costly projects where failure would cost a fortune.	• Proactive mitigation: Catches and fixes big problems before moving to the next stage. • Balanced flexibility: Lowers risks with quick test models while still welcoming changes. • Scale adaptability: Handles giant, complex projects easily as new technical hurdles appear..	• Expert-dependent: Needs skilled risk experts to work properly. • Costly and complex: Too expensive and heavy for small projects.. • Endless loops: Can get stuck repeating cycles if risks aren't controlled tightly.	• Huge, risky projects: Giant, expensive systems where failure would be a total disaster. • Unproven technology: Projects using brand-new tech that must be tested early to see if it even works. • Mission-critical systems: Life-or-death systems that combine many outside teams and complex moving parts.
Rational Unified Process (RUP)	• Four business phases: Divides work into Inception, Elaboration, Construction, and Transition • Separates business from tech: Keeps management stages apart from daily coding and testing. • Combines loops and diagrams: Mixes step-by-step building with UML visual design.	• Flexible structure: Separates big business planning from daily coding and testing. • Component focus: Focuses heavily on building clean parts and checking code quality. • Dedicated launch: Treats handing over and launching the software as its own full step.	• Too heavy: Too complex and bulky for simple projects. • Not for everything: Does not work well for tiny systems like embedded chips. • Needs extra training: Teams need deep training to understand and draw UML diagrams.	• Big company systems: Giant apps that need strong structure and step-by-step releases. • Multi-team projects: Shared systems that use UML diagrams to keep everyone aligned. • Complex business apps: Big software needing clear steps from planning to launch.